МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 1

з дисципліни “Професійний Python”

на тему “Організація професійного Python-проєкту та Git “

Виконала

студентка 3 курсу

групи ФЕП-33

Бойчук Карина Іванівна

Перевірив

Свелеба Сергій Андрійович

Львів – 2026

Мета роботи: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Хід роботи:

Варіант 2. Система обліку книг

Для виконання лабораторної роботи було використано Python версії:

```
PS D:\3\python\Lab_1> py --version
Python 3.14.5
PS D:\3\python\Lab_1>
```

Було створено каталог проєкту, у якому розміщено всі файли консольного застосунку:

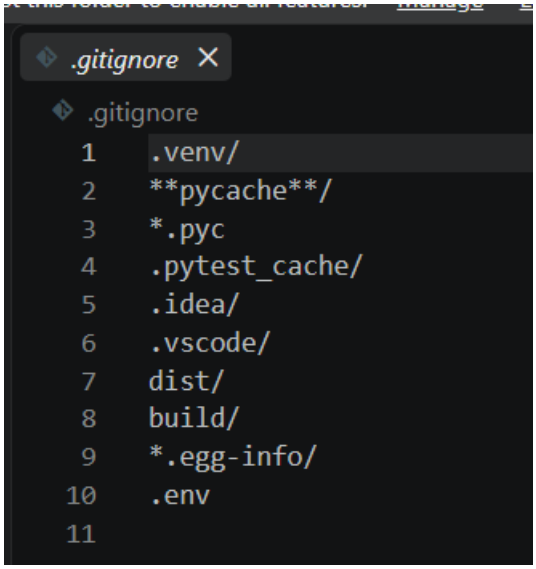
```
            pythonw.exe

├──src
│   ├──book_manager
│   │       main.py
│   │       models.py
│   │       services.py
│   │       __init__.py
│   │
│   │   └──__pycache__
│   │           main.cpython-314.pyc
│   │           models.cpython-314.pyc
│   │           services.cpython-314.pyc
│   │           __init__.cpython-314.pyc
│   │
│   └──book_manager.egg-info
│           dependency_links.txt
│           entry_points.txt
│           PKG-INFO
│           SOURCES.txt
│           top_level.txt
│
└──tests
        test_services.py
```

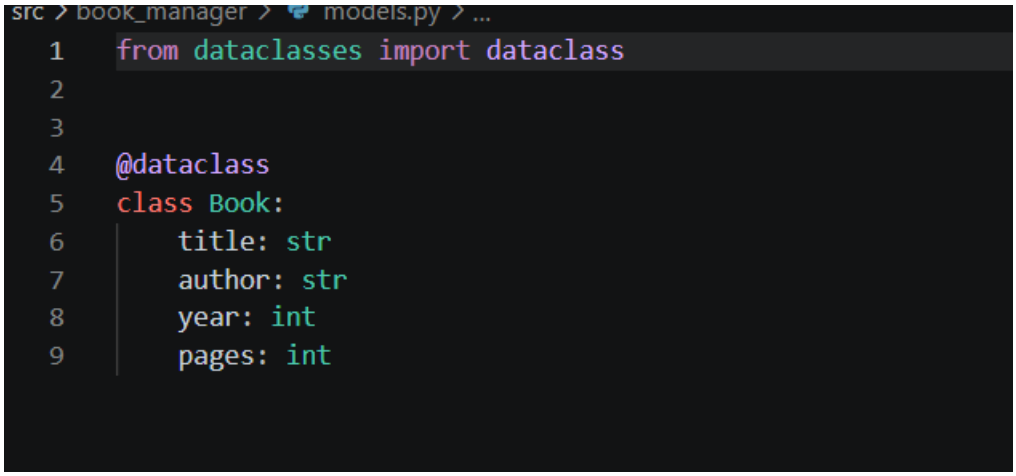
Створення .venv:

```
PS D:\3\python\Lab_1> .venv\Scripts\Activate.ps1
(.venv) PS D:\3\python\Lab_1>
```

Для виключення службових файлів та директорій із Git-репозиторію було створено файл .gitignore:



Для представлення книги було створено модель Book. Для її реалізації використано dataclass. Модель містить назву книги, автора, рік видання та кількість сторінок:



Основну бізнес-логіку застосунку винесено в окремий модуль services.py. У ньому реалізовано додавання книг, пошук за автором, фільтрацію за роком, пошук найбільшої книги, обчислення середньої кількості сторінок та сортування:

```
from book_manager.models import Book

def add_book(
    books: list[Book],
    title: str,
    author: str,
    year: int,
    pages: int,
) -> Book:
    book = Book(
        title=title,
```

```
        author=author,
        year=year,
        pages=pages,
    )
    books.append(book)
    return book

def find_by_author(
    books: list[Book],
    author: str,
) -> list[Book]:
    return [
        book
        for book in books
        if book.author.lower() == author.lower()
    ]

def filter_by_year(
    books: list[Book],
    year: int,
) -> list[Book]:
    return [
        book
        for book in books
        if book.year == year
    ]


def find_largest_book(
    books: list[Book],
) -> Book | None:
    if not books:
        return None

    return max(
        books,
        key=lambda book: book.pages,
    )


def calculate_average_pages(
    books: list[Book],
) -> float:
    if not books:
        return 0.0

    total_pages = sum(
```

```
        book.pages
        for book in books
    )

    return total_pages / len(books)


def sort_books_by_pages(
    books: list[Book],
) -> list[Book]:
    return sorted(
        books,
        key=lambda book: book.pages,
        reverse=True,
    )
```

Файл main.py є точкою входу в програму. У ньому реалізовано консольне меню, введення даних користувачем та виведення результатів роботи програми:

```
from book_manager.models import Book
from book_manager.services import (
    add_book,
    calculate_average_pages,
    filter_by_year,
    find_by_author,
    find_largest_book,
    sort_books_by_pages,
)


def create_demo_books() -> list[Book]:
    books: list[Book] = []

    add_book(
        books,
        "Kobzar",
        "Taras Shevchenko",
        1840,
        256,
    )

    add_book(
        books,
        "Harry Potter",
        "J. K. Rowling",
        1997,
        320,
```

```
    )

    add_book(
        books,
        "The Hobbit",
        "J. R. R. Tolkien",
        1937,
        310,
    )

    add_book(
        books,
        "1984",
        "George Orwell",
        1949,
        328,
    )

    return books


def print_books(books: list[Book]) -> None:
    if not books:
        print("\nBooks not found.")
        return

    print("\nBooks:")

    for book in books:
        print(
            f"{book.title:20}"
            f"{book.author:25}"
            f"{book.year:8}"
            f"{book.pages:6}"
        )


def input_year() -> int:
    while True:
        try:
            year = int(input("Enter publication year: "))

            if year < 0:
                print("Year cannot be negative.")
                continue

            return year
```

```
        except ValueError:
            print("Please enter a number.")


def input_pages() -> int:
    while True:
        try:
            pages = int(input("Enter number of pages: "))

            if pages <= 0:
                print("Number of pages must be positive.")
                continue

            return pages

        except ValueError:
            print("Please enter a number.")


def add_book_from_input(
    books: list[Book],
) -> None:
    title = input("Enter book title: ").strip()
    author = input("Enter author: ").strip()
    year = input_year()
    pages = input_pages()

    if not title or not author:
        print("Title and author cannot be empty.")
        return

    add_book(
        books,
        title,
        author,
        year,
        pages,
    )

    print("Book added successfully.")


def print_menu() -> None:
    print("\n" + "=" * 45)
    print("BOOK MANAGER")
    print("=" * 45)
```

```
    print("1. Show all books")
    print("2. Add book")
    print("3. Find books by author")
    print("4. Filter books by year")
    print("5. Find the largest book")
    print("6. Calculate average number of pages")
    print("7. Sort books by pages")
    print("8. Exit")
    print("=" * 45)


def run_menu(books: list[Book]) -> None:
    while True:
        print_menu()

        command = input("Select command: ").strip()

        if command == "1":
            print_books(books)

        elif command == "2":
            add_book_from_input(books)

        elif command == "3":
            author = input("Enter author: ").strip()

            result = find_by_author(
                books,
                author,
            )

            print_books(result)

        elif command == "4":
            year = input_year()

            result = filter_by_year(
                books,
                year,
            )

            print_books(result)

        elif command == "5":
            largest = find_largest_book(books)

            if largest is not None:
```

```
            print("\nLargest book:")
            print(
                f"Title: {largest.title}"
            )
            print(
                f"Author: {largest.author}"
            )
            print(
                f"Year: {largest.year}"
            )
            print(
                f"Pages: {largest.pages}"
            )
        else:
            print("\nNo books available.")

    elif command == "6":
        average = calculate_average_pages(books)

        print(
            f"\nAverage number of pages: "
            f"{average:.2f}"
        )

    elif command == "7":
        sorted_books = sort_books_by_pages(books)

        print("\nBooks sorted by number of pages:")
        print_books(sorted_books)

    elif command == "8":
        print("\nGoodbye!")
        break

    else:
        print("\nUnknown command.")


def main() -> None:
    books = create_demo_books()

    print("Welcome to Book Manager!")
    run_menu(books)


if __name__ == "__main__":
    main()
```

Встановлення package у editable mode:

```
(.venv) PS D:\3\python\Lab_1> python -m pip install -e .
Obtaining file:///D:/3/python/Lab_1
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: book-manager
  Building editable for book-manager (pyproject.toml) ... done
  Created wheel for book-manager: filename=book_manager-0.1.0-0.editable-py3-none-any.whl size=1537 sha256=972f58db31bcd3694efcffc757
ade1b9606d3bcbf
  Stored in directory: C:\Users\karin\AppData\Local\Temp\pip-ephem-wheel-cache-z1leczrp\wheels\51\65\81\bc18e7b0c123f0872d29c52ce83c6
Successfully built book-manager
Installing collected packages: book-manager
  Attempting uninstall: book-manager
    Found existing installation: book-manager 0.1.0
    Uninstalling book-manager-0.1.0:
      Successfully uninstalled book-manager-0.1.0
Successfully installed book-manager-0.1.0

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\3\python\Lab_1>
```

Перевірка запуску програми:

```
(.venv) PS D:\3\python\Lab_1> python -m book_manager.main
Welcome to Book Manager!

==========================================
BOOK MANAGER
==========================================
1. Show all books
2. Add book
3. Find books by author
4. Filter books by year
5. Find the largest book
6. Calculate average number of pages
7. Sort books by pages
8. Exit
==========================================
Select command:
```

Приклад з одної команди:

```
==========================================
Select command: 1

Books:
Kobzar              Taras Shevchenko            1840   256
Harry Potter        J. K. Rowling               1997   320
The Hobbit          J. R. R. Tolkien            1937   310
1984                George Orwell               1949   328
```

Тестування:

```
(.venv) PS D:\3\python\Lab_1> python -m pytest
============================= test session starts =============================
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\3\python\Lab_1
configfile: pyproject.toml
collected 6 items

tests\test_services.py ......
```

README.md:

```
**# Book Manager**
```

Laboratory project for the course "Professional Python".

Description

Book Manager is a console Python application for managing a collection of books.

For each book the program stores:

*** title;**

*** author;**

*** publication year;**

*** number of pages.**

The program supports:

*** adding a book;**

*** searching books by author;**

*** filtering books by publication year;**

*** finding the largest book;**

*** calculating the average number of pages;**

*** sorting books by number of pages.**

Requirements

Python 3.11+

Installation

Create a virtual environment:

```text

python -m venv .venv

```

Activate the virtual environment.

Install the project:

```text

python -m pip install -e .

```

Run

Run as a module:

```text

python -m book_manager.main

```

or use the console command:

```text

book-manager

```

Project structure

```text

src/book_manager/models.py

```

Data model for books.

```text

src/book_manager/services.py

```

Business logic of the application.

```text

src/book_manager/main.py

```

Application entry point and console menu.

Example

```text

BOOK MANAGER

==========

1. Show all books

2. Add book

3. Find books by author

4. Filter books by year

5. Find the largest book

6. Calculate average number of pages

7. Sort books by pages

8. Exit

==========

Select command: 5

Largest book:

Title: 1984

Author: George Orwell

Year: 1949

```
**Pages: 328**

**```**



**## Author**



**Student: Boichuk Karyna**



**Group: FEP-33**
```

pyproject.toml:

D:\3\python\Lab_1\src\book_manager\services.py

```toml
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "book-manager"
version = "0.1.0"
description = "Book management laboratory project"
requires-python = ">=3.11"

[project.scripts]
book-manager = "book_manager.main:main"

```

Висновок: У ході лабораторної роботи було створено консольний Python-застосунок «Система обліку книг» відповідно до індивідуального варіанта. Було використано структуру src-layout, створено модель даних Book, окремий модуль бізнес-логіки та головний файл main.py. У програмі реалізовано додавання книг, пошук за автором, фільтрацію за роком видання, визначення найбільшої книги, обчислення середньої кількості сторінок та сортування. Також було використано type hints, dataclass, віртуальне середовище, pyproject.toml, README.md та .gitignore. Для перевірки роботи бізнес-логіки було створено тести, які

успішно виконуються. Проєкт також було додано до Git-репозиторію та створено необхідну кількість комітів.